

ADA · Change Detection — User Manual

Welcome, and thank you for choosing the **ADA Encroachment Change-Detection Service**. This manual has been prepared to guide you — whether you are an operator deploying the platform for the first time, an analyst running detections, or a stakeholder reviewing results — through every aspect of the service in a clear and structured manner. We invite you to read the sections most relevant to your role; each has been written to stand on its own while forming part of a complete operational reference.

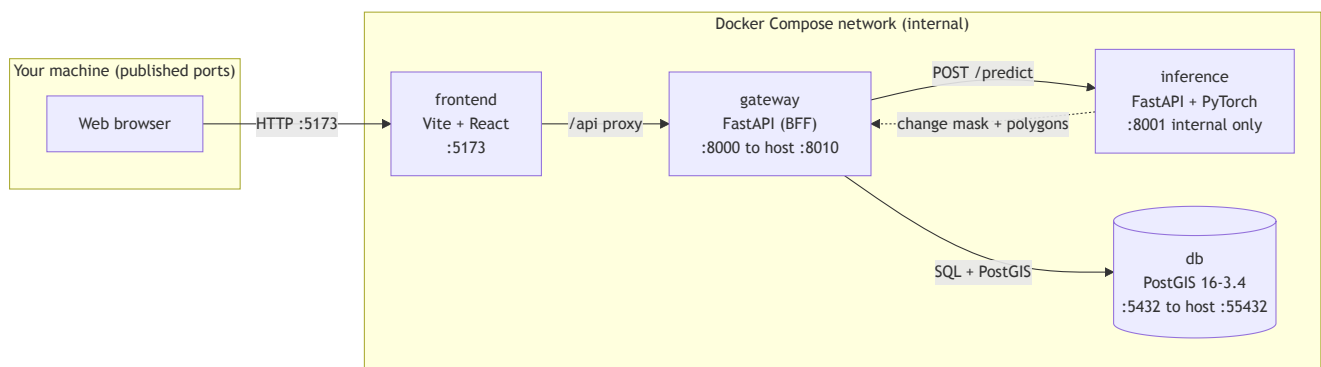
The service compares two orthophotos of the same location captured at different times (a *before* image, **T1**, and an *after* image, **T2**), identifies areas of physical change such as new construction or encroachment, and returns georeferenced polygons annotated with surface area and confidence. Results are persisted to a spatial database and presented on an interactive map.

Table of Contents

1. [Getting Started — Prerequisites, Configuration & Launch](#)
2. [End-to-End Process Flow](#)
3. [Technical Models Implemented in the Service](#)
4. [Collaboration, Visibility & Stakeholder Access](#)
5. [Expansion Roadmap](#)

System Architecture at a Glance

Before proceeding, the following diagram orients you to the four cooperating microservices and their boundaries. Only the **frontend** and the **gateway** are exposed to your machine; the **inference** engine and the **database** remain private on the internal container network.



Note on Mermaid rendering. Every diagram in this manual is authored as a fenced ````mermaid` block. These render automatically in GitHub, GitLab, VS Code (with the Markdown Preview Mermaid extension), Obsidian, and any Mermaid-aware viewer. If you are reading this in a plain-text editor, paste the block contents into the [Mermaid Live Editor](#) to view the rendered figure.

1. Getting Started — Prerequisites, Configuration & Launch

This section walks you through everything required to bring the service online, from verifying your environment to confirming a healthy deployment.

1.1 Prerequisite Setup Requirements

Please ensure the following are installed and available on your host machine before proceeding:

Requirement	Minimum	Purpose
Docker Engine	24.x	Builds and runs all four service containers
Docker Compose	v2 (the <code>docker compose</code> plugin)	Orchestrates the multi-service stack
GNU Make	any recent	Convenience wrapper for the launch commands
Free disk space	~4 GB	Container images (PyTorch, GDAL) and the model weights
Available host ports	5173, 8010, 55432	Frontend, gateway, and database respectively

The service adopts a deliberate **offline, non-China weight posture**: the only pretrained artifact it downloads is the torchvision **ResNet34 ImageNet** encoder (`resnet34-b627a593.pth`), retrieved from `download.pytorch.org` . This file is cached into the image at build time so that the first inference request is warm and fully offline thereafter.

1.2 System Configuration Checks

1. **Confirm Docker is running.** Execute `docker ps` ; the command should return without error.
2. **Verify port availability.** The stack publishes host ports **5173** (frontend), **8010** (gateway), and **55432** (database). If any are already occupied by another stack, override them via the environment before launch — the gateway port, for example, honours `GATEWAY_HOST_PORT` , and the database honours `POSTGRES_HOST_PORT` .

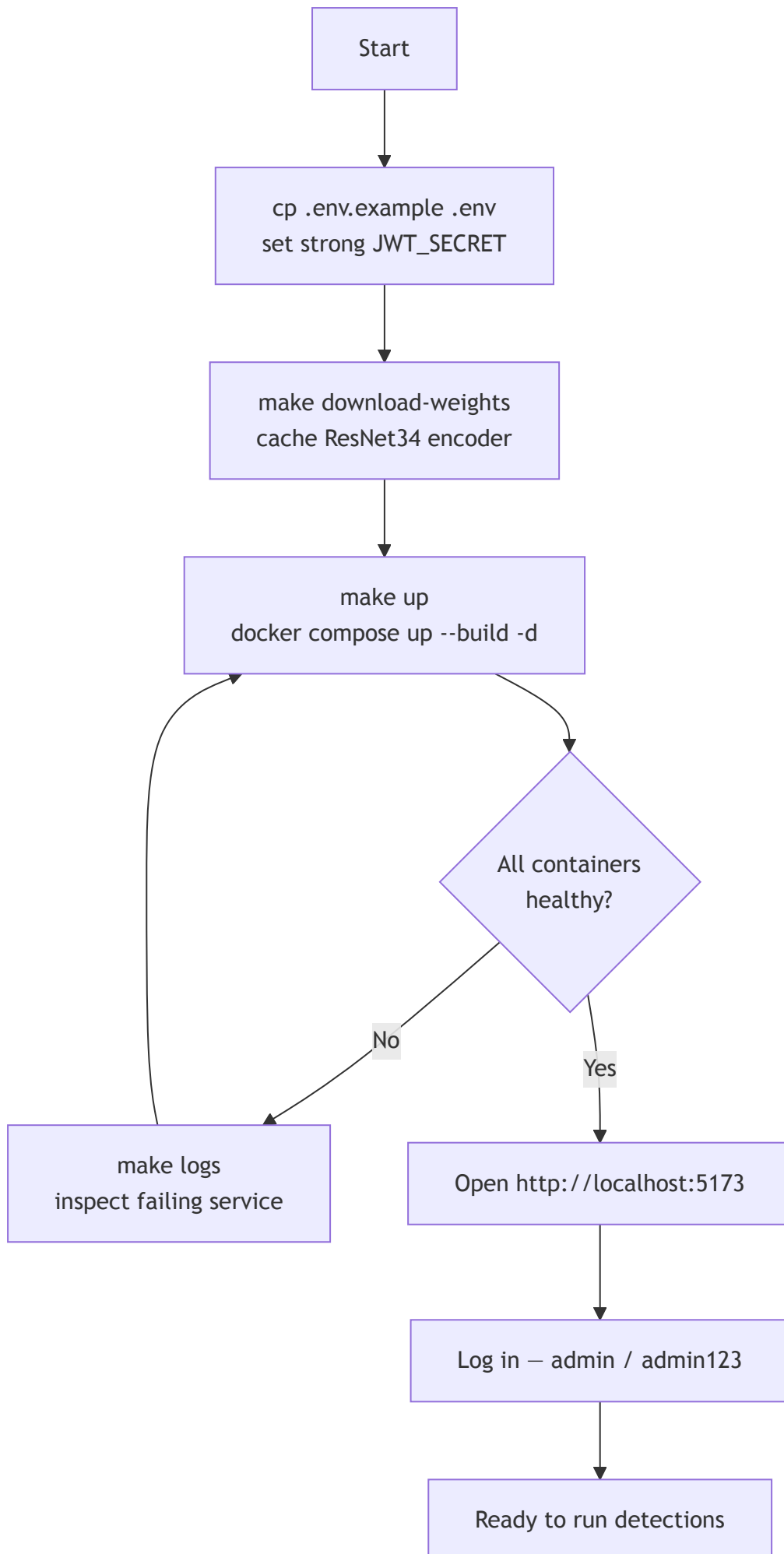
3. **Review the environment file.** Copy the template and set a strong secret:

```
cp .env.example .env
```

At minimum, replace `JWT_SECRET` with a 64-character random hex string before any non-local use. The remaining defaults (`POSTGRES_USER` , `POSTGRES_DB` , `CORS_ORIGINS` , `INFERENCE_URL`) are pre-configured for local operation.

1.3 Launch Command Workflow

The following diagram summarises the launch sequence and its verification gates.



Executed as commands:

```
# 1. Pre-fetch the pretrained encoder (idempotent; skips if already cached)
make download-weights

# 2. Build all images and start the stack in the background
make up

# 3. Tail logs if you wish to watch startup
make logs

# 4. Run the end-to-end smoke test against the running stack (optional)
make test
```

1.4 Confirming a Successful Deployment

- **Container status:** `docker compose ps` should list `db`, `inference`, and `gateway` as **healthy**, and `frontend` as **up**.
- **Gateway health probe:** a request to the gateway health endpoint should return `{"status":"ok"}`.
- **Application access:** navigate to <http://localhost:5173> and sign in with the seeded demonstration credentials `admin` / `admin123`.

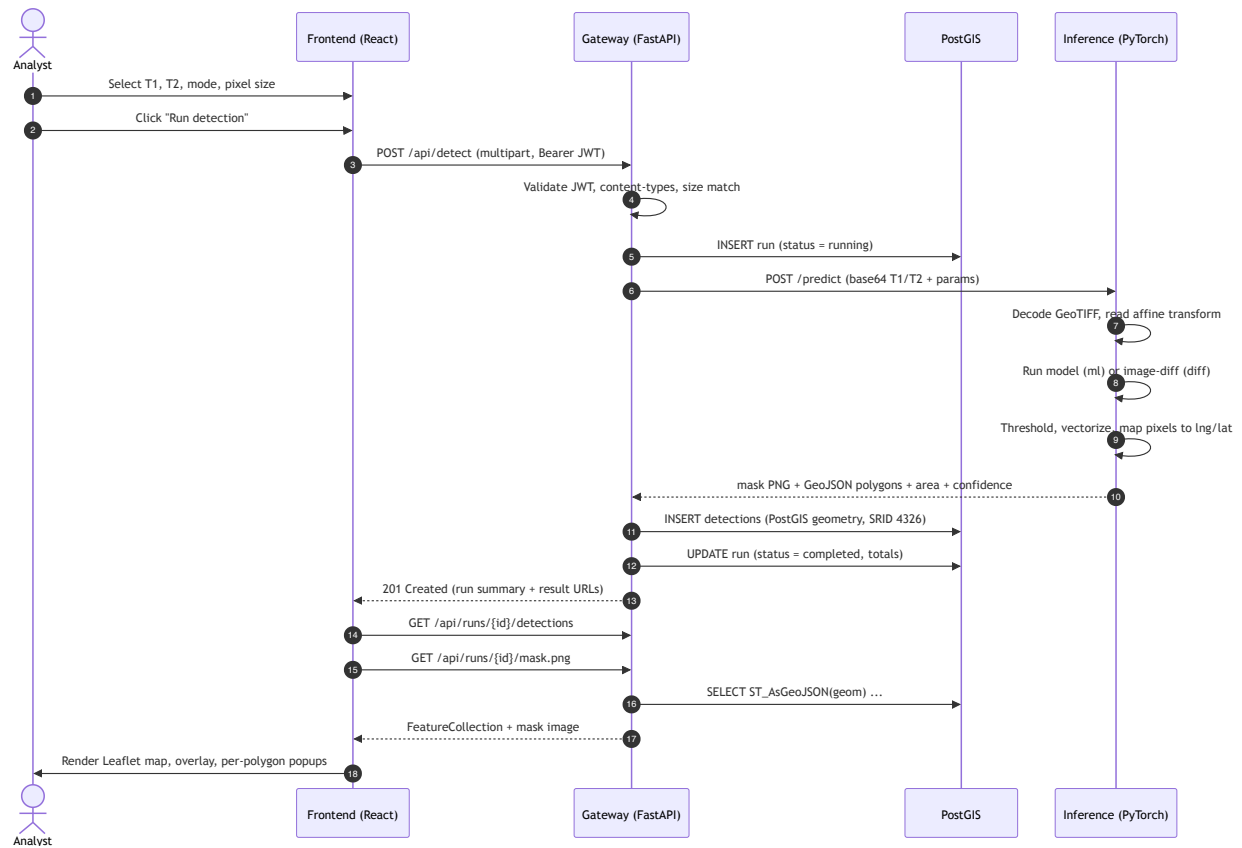
Security advisory. The `admin/admin123` account and the development `JWT_SECRET` are intended solely for local evaluation. Rotate both before deploying to any shared or production environment.

1.5 Stopping and Resetting

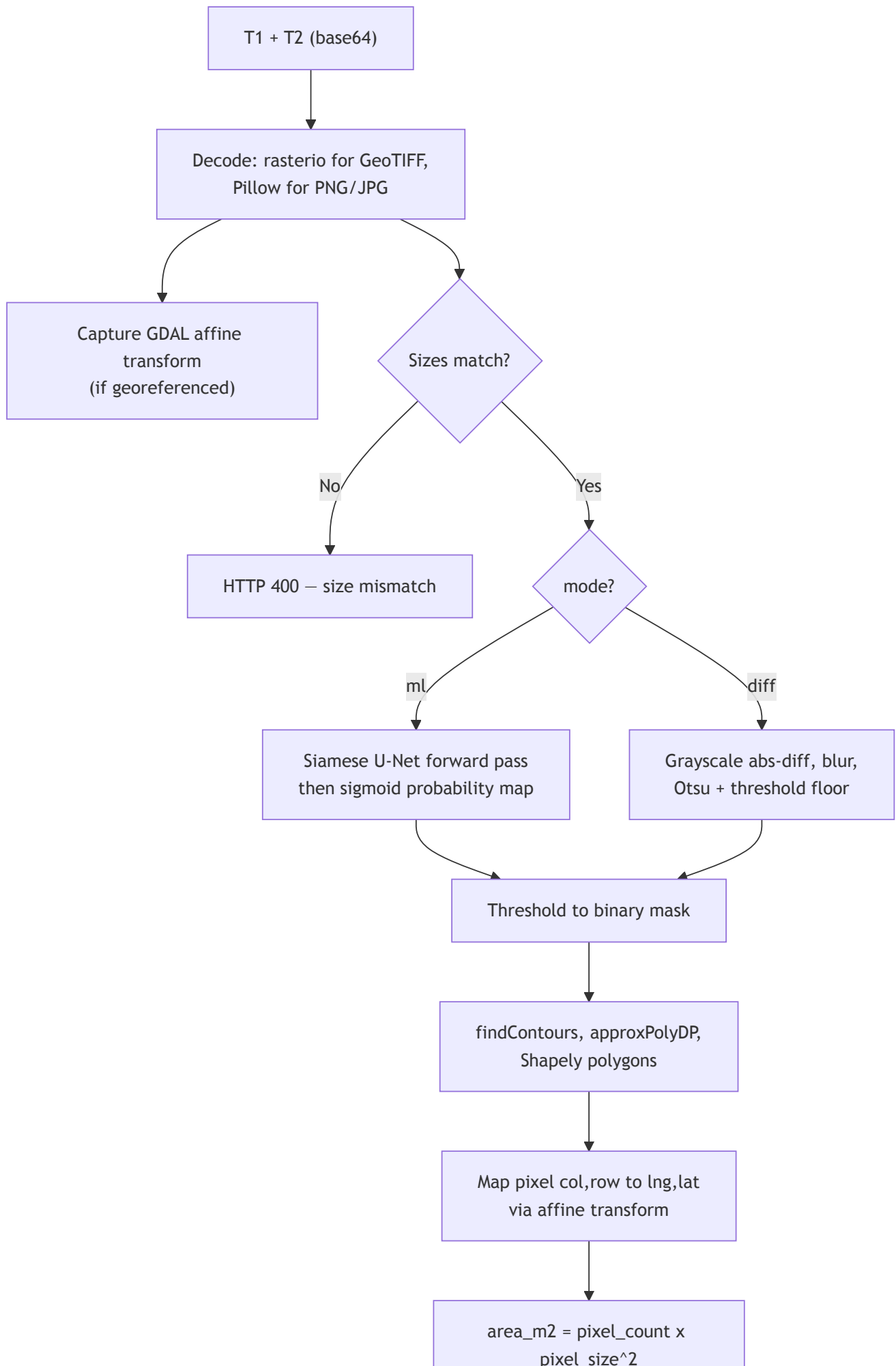
Command	Effect
<code>make down</code>	Stops the stack, preserving the database volume
<code>make clean</code>	Stops the stack and deletes volumes (wipes the database)

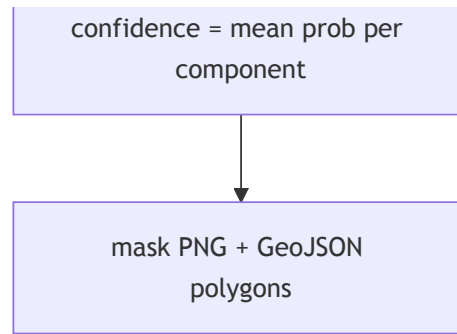
2. End-to-End Process Flow

This section presents the complete sequence of a detection, from the moment you select a pair of images to the rendering of georeferenced change polygons on the map. The diagram traces every hop across the four services.



Internally, the inference engine follows the pipeline below. This is the heart of the *"internal system workflow"* — the transformation of two rasters into vector geometry.



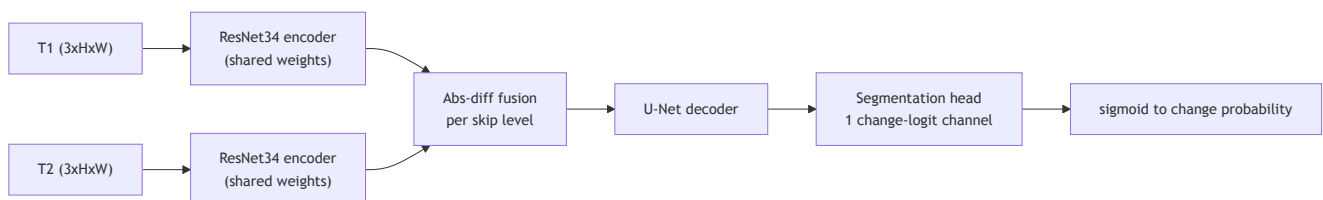


3. Technical Models Implemented in the Service

The service implements **two interchangeable change-detection models** plus a supporting **pretrained encoder** and a **spatial persistence model**. Each is described below with its function, use cases, and unique role in the architecture.

3.1 Siamese U-Net — the ML Change Model (`mode = ml`)

Core functionality. This is a bitemporal **FC-Siam-diff**-style network. A single, shared-weight **ResNet34** encoder processes T1 and T2 independently. At every skip level, the two feature maps are fused by **absolute difference**, $|f(T1) - f(T2)|$, which is channel-preserving so that a stock U-Net decoder consumes them unchanged. The decoder produces a single **change-logit** channel (B, 1, H, W); a sigmoid applied downstream turns logits into a per-pixel change probability.



Use cases. Nuanced, semantically-aware detection where naïve pixel differencing would be fooled by illumination, seasonal, or radiometric variation — for example distinguishing genuine new construction from lighting change across two acquisition dates.

Unique purpose. It is the platform's flagship, learning-based detector. Its accuracy depends on a trained *change head*; the shared encoder provides transferable low-level features, while the trainable decoder specialises in the change signal.

Operational note. In this MVP build the change head ships **untrained**. The service boots successfully with a random head so that the platform and `diff` mode remain fully available, but `ml`-mode masks will be noisy until a head is trained (see §5). The `/health` endpoint reports whether trained weights were loaded.

3.2 Image-Differencing Model — the Non-ML Fallback (`mode = diff`)

Core functionality. A classical computer-vision pipeline: convert both images to grayscale, compute an **absolute difference**, apply a Gaussian blur to suppress noise, normalise the result to a pseudo-probability map, then binarise using **Otsu's method** combined with a caller-supplied threshold floor.

Use cases. Immediate, deterministic results that require **no trained model** — ideal for demonstrations, smoke testing, quick-look analysis, and any deployment where a trained change head is not yet available.

Unique purpose. It guarantees the service is *useful on day one*. Crucially, it reuses the **identical vectorization, area, and confidence logic** as the ML path, so both modes emit results of the same shape and semantics — the map, database, and API cannot tell which model produced a given polygon.

3.3 ResNet34 ImageNet Encoder — the Pretrained Backbone

Core functionality. The torchvision **ResNet34** convolutional backbone, pretrained on ImageNet (`IMAGENET1K_V1`), supplies the feature extractor embedded within the Siamese U-Net.

Use cases. Transfer learning — the encoder's general-purpose visual features accelerate and stabilise training of the change head on comparatively small orthophoto datasets.

Unique purpose. It is the sole pretrained weight in the system and is loaded **offline and safely**: from a local copy when present, and always via `weights_only` semantics (or safetensors) so that no executable pickle is ever deserialised. This underpins the service's security and non-China supply-chain posture.

3.4 Spatial Persistence Model — PostGIS Geometry

Core functionality. Each detected polygon is stored as a PostGIS `geometry(Polygon, 4326)` value alongside its `area_m2`, `confidence`, and `class_label`, and is served back to clients via `ST_AsGeoJSON`. A GiST spatial index accelerates geometric queries.

Use cases. Durable, queryable storage of results; spatial analysis; and standards-compliant GeoJSON delivery to the Leaflet map.

Unique purpose. It is the system of record. By persisting geometry in a true spatial database rather than opaque blobs, the service keeps results open to downstream GIS tooling and future spatial analytics.

4. Collaboration, Visibility & Stakeholder Access

This section explains how features, statuses, and outputs are surfaced to team members and stakeholders during collaborative development and active use.

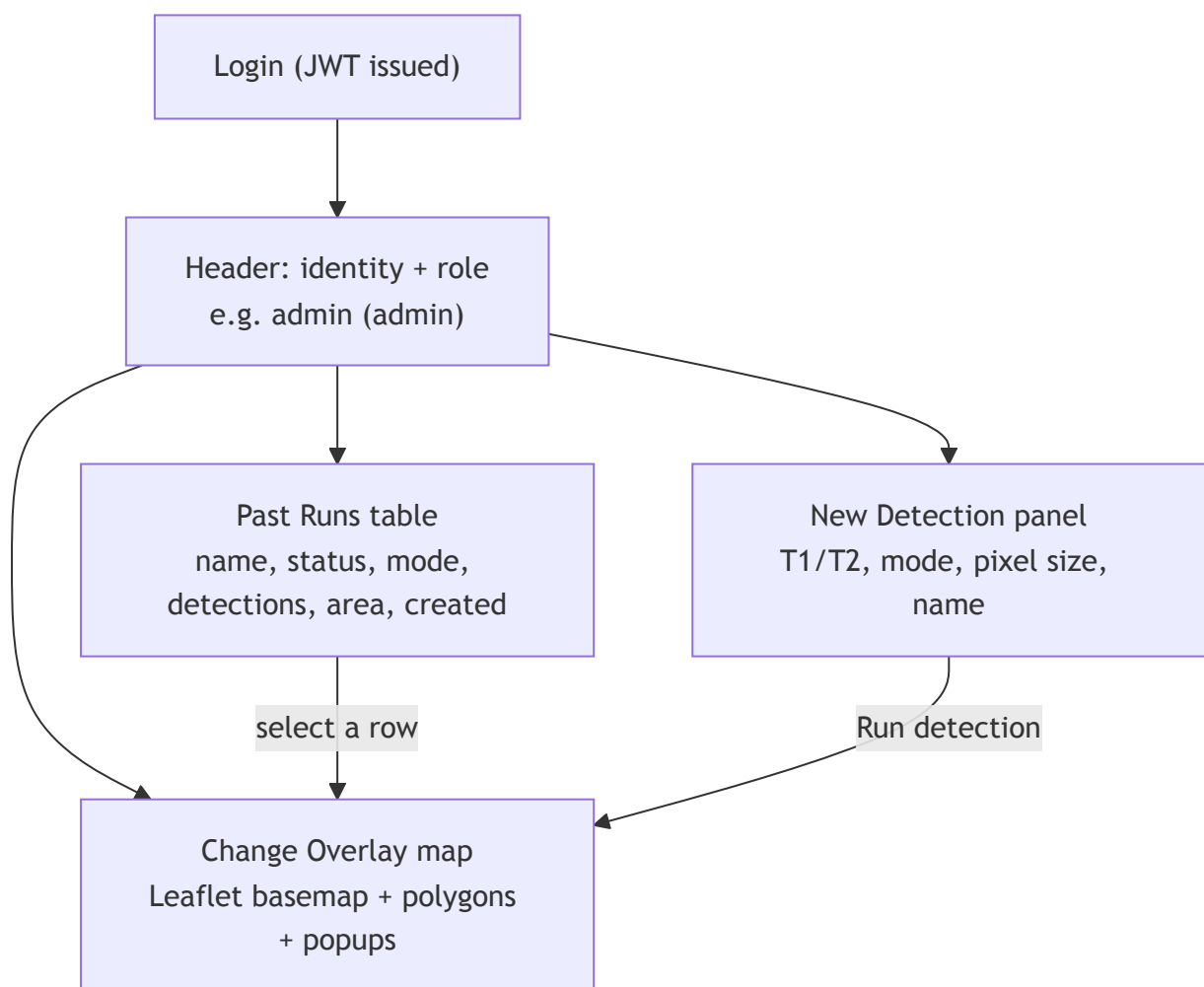
4.1 Access Permissions & Roles

Authentication is handled by the gateway using **HS256 JSON Web Tokens**. Passwords are stored as bcrypt hashes; a token is issued at login and must accompany every subsequent API call as a `Bearer` credential.

Role	Default	Capabilities in the current build
<code>admin</code>	Seeded demo account	Full access to the application; runs are owned by the account that creates them
<code>analyst</code>	Default for new users	Create and view their own runs and detections

Every run is **scoped to its owner** by `user_id`. When a client requests a run, its detections, or its mask, the gateway verifies ownership before responding, returning **404 Not Found** for runs belonging to other users. This provides tenant-style isolation between team members today; role-differentiated and shared-visibility access is on the roadmap (see §5).

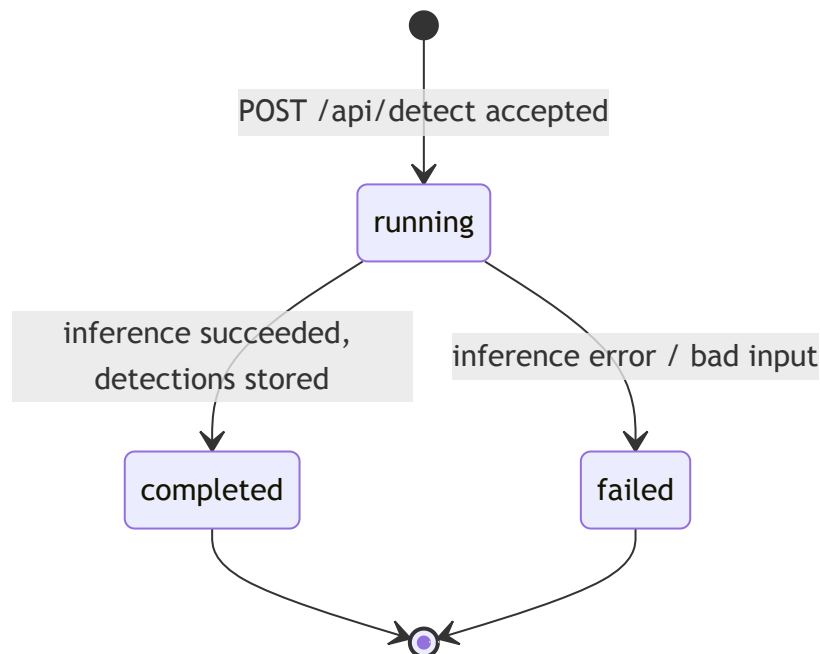
4.2 Interface Visibility — What Each User Sees



- **New Detection panel** — the controls for launching a run: T1/T2 file pickers, mode selector (`ml` / `diff`), pixel size, and an optional run label.
- **Change Overlay map** — an interactive Leaflet map that renders the georeferenced change polygons over an OpenStreetMap basemap, with a translucent mask overlay and clickable popups reporting each detection's **area (m²)** and **confidence**. When inputs are not georeferenced, the interface gracefully falls back to displaying the raw change-mask image.
- **Past Runs table** — a paginated history showing each run's **name, status, mode, detection count, total area, and creation time**, allowing any past run to be re-selected and re-visualised.

4.3 Operational Statuses

Each run advances through an explicit lifecycle that is visible in the Past Runs table and via the API:



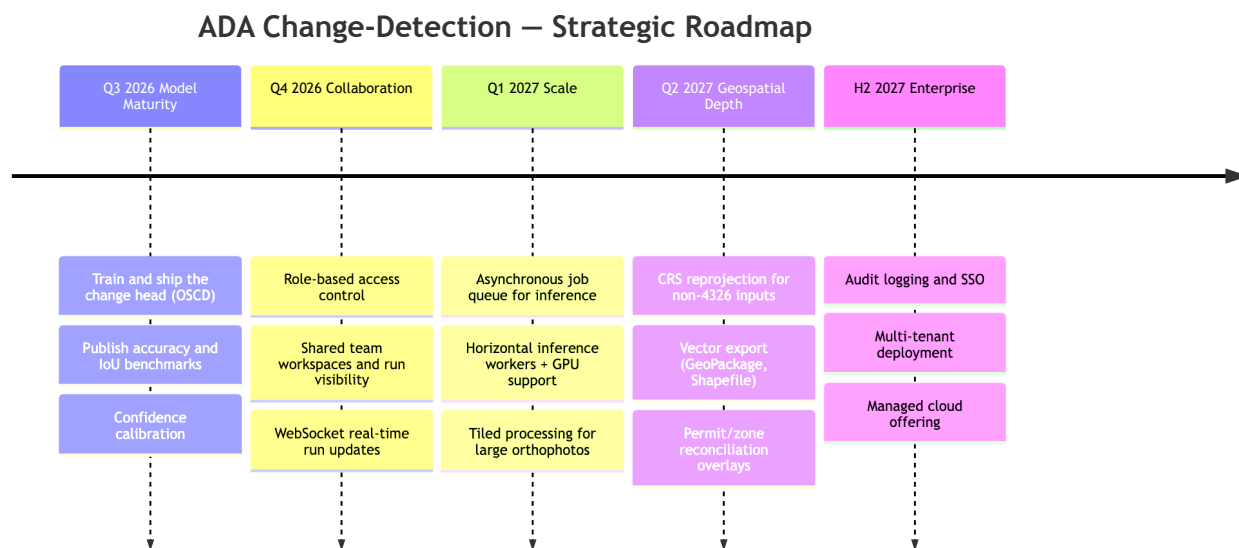
Service health is independently observable through the gateway and inference **health endpoints**, which report liveness, model-load status, and the active compute device — valuable signals for developers and operators monitoring a shared deployment.

4.4 Real-Time Update Protocol

The current interface follows a **request/refresh** model: upon completion of a detection, the frontend immediately fetches the new run's detections and mask and updates the map, while the Past Runs list reflects the latest state on load and navigation. This keeps all collaborators viewing consistent, database-backed results. A push-based live-update channel is a planned enhancement (see §5).

5. Expansion Roadmap

We are committed to evolving the service from a robust MVP into a production-grade platform. The following roadmap outlines planned feature enhancements, scalability improvements, technology upgrades, and strategic milestones.



5.1 Planned Feature Enhancements

- **Train and deliver the ML change head** so that `ml` mode produces accurate, calibrated masks, complete with published benchmarks.
- **Vector and report export** — download detections as GeoPackage/Shapefile and generate PDF encroachment reports.
- **Permit and zoning reconciliation** — overlay detected change against authorised-construction records to flag likely encroachments automatically.

5.2 Scalability Improvements

- **Asynchronous inference** via a job queue, decoupling long-running predictions from the request cycle.
- **Horizontal inference workers** with optional **GPU acceleration** for throughput.
- **Tiled processing** so that large, high-resolution orthophotos are handled without memory pressure.

5.3 Technology Stack Upgrades

- **CRS reprojection** for inputs in projections other than EPSG:4326, extending correct georeferencing to any coordinate system.
- **Push-based updates** through WebSockets, replacing request/refresh with live status streaming.

- **Observability** — structured logging, metrics, and tracing across all services.

5.4 Long-Term Strategic Milestones

- **Role-based access control** and **shared team workspaces**, evolving today's per-user isolation into collaborative, permission-differentiated visibility.
- **Enterprise readiness** — single sign-on, audit trails, and multi-tenant isolation.
- **A managed cloud offering**, delivering the platform as a hosted service.

Thank you for using the ADA Change-Detection Service. Should you encounter difficulty, begin with `make logs` to inspect service output, verify the health endpoints, and confirm that host ports 5173, 8010, and 55432 are free. We wish you productive and accurate detections.